Biggy - Submission Write-Up

What I built

Biggy turns a vague incident report into a briefing you can trust. You give it a sentence ("checkout is throwing 504s") and it runs a bounded, hypothesis-driven investigation over a workspace of operational evidence, producing a grounded briefing: ranked hypotheses, red herrings ruled out *with a reason*, a recommended action, and a machine-verified grounding score.

The goal is to earn an engineer's trust by making every claim traceable and machine-verifiable (grounding), keeping the agent's confidence honest (calibration), and making the reasoning visible (transparency).

The architecture I chose

Biggy is mostly a workflow - a fixed five-phase pipeline - with one agentic step (the bounded investigate loop). It runs on a deliberately cheap model, `gemini-3.1-flash-lite`, and produces promising results which are comparable to the frontier evals.

The rest of the stack is plain-Python orchestration, LangChain, Pydantic schemas at every LLM boundary, and versioned prompt files. Failure handling is built in: exponential backoff, structured-output repair, and tool errors fed back to the agent.

The workflow is initially deployed as a standalone library wrapped into a CLI. The same library is also wrapped into a NextJS web app (with a python worker, using Redis for job tracking).

How the agentic flow works

A five-phase pipeline built on abductive reasoning: rather than read everything and summarize, it forms competing theories and aims its search at the evidence that would *kill* them. The first three phases are LLM judgment; the last two are deterministic code.

- **Hypothesize:** One structured LLM call, *no tools* - seeded with the symptom, the evidence *manifest* (what exists, not its contents), and the changes that landed. It proposes a small candidate set including the "obvious" suspect, each with a concrete disconfirming test. Human signal (Slack, tickets) is held back, so the slate comes from what *changed*, not hearsay.
- **Investigate (the bounded loop):** The LLM works through those tests on a read-only tool budget (see available tools below), seeking evidence that proves each hypothesis *wrong*. Where a free-form ReAct agent rationalizes its first guess, here a hypothesis only survives if it was attacked and held.
- **Adjudicate:** The agent ranks the hypotheses with confidence capped in code. The schema also supports an explicit `inconclusive` verdict, and this is the one calibration behaviour today's models don't reliably deliver: on the deliberately balanced case, both the default *and* Gemini pro-tier model over-commit to a single cause instead of holding the ~50/50 split. So calibration is real but partial – this requires further work (prompt engineering, or other classes of models).

- **Verify (the hard floor):** A deterministic pass re-opens every cited source and confirms the quoted snippet exists; a missing one is flagged, and the grounding badge drops.
- **Reconcile:** A final no-LLM pass flags the public status draft when it still blames a cause the evidence doesn't support, and summarizes customer impact from in-window tickets.

The tools (the agent's hands)

Six read-only time clamped tools are exposed to the investigate loop. Every result carries the `path:line` it came from, so a citation can only point at evidence returned. Structured accessors (`get_topology`, `get_changes`, `get_metric`) parse data into clean facts; raw tools (`read_file`, `search`) handle unstructured logs and chat.

Tool	Returns
`list_evidence()`	the manifest - what evidence exists and the time ranges it covers
`read_file(path)`	one evidence file, line-numbered, auto-sliced to the incident window
`search(keyword)`	keyword hits across in-window evidence, as `path:line: text`
`get_topology(service)`	a service's upstream `depends_on`, derived dependents, and shared infrastructure
`get_changes(service?)`	deploys / config changes / migrations in the window (and their rollbacks)
`get_metric(name)`	a metric time-series with peak / min / max, for timing correlation

There is no embedding RAG *inside* a run - retrieval is these source-attached tools, and the only graphs are lightweight (in-memory topology + the ledger's hypothesis↔evidence structure). Semantic/vector cross-incident recall exists in the corpus but is a deferred follow-up.

How memory/state is handled

State lives in a single, evolving, serializable Investigation Ledger that every phase reads and mutates - working memory during the run & returned in the CLI & web platform after. The investigation's narrative *is* the ledger's record of hypotheses, tool calls, verdict, grounding, and comms checks as hypotheses.

How I built it (AI-assisted workflow)

Built with Claude Code in tight, eval-gated loops, proving the reasoning before any UI.

1. **Synthetic world first.** A time-indexed ~20-service e-commerce corpus - standing knowledge (topology, runbooks, ADRs, ~10 post-mortems), windowed telemetry, and messy human signal - across seven scenario seeds, each clamped to its incident window so the agent can't read the future.
2. **CLI-First Scaffold:** Started by building a pure CLI application to nail the fundamentals - parsing the synthetic data, handling timestamps, and building the basic execution loop.
3. **The Agent Layer:** Focused entirely on the reasoning engine, iterating on the prompt design, the bounded tool loop, failure handling, and the deterministic verification pass.
4. **The Evaluation Harness:** `biggy eval` regression-tests the current shipped cases - it runs the engine blind, then grades the result against a hidden answer key.

5. **Library & Worker Decoupling:** Wrapped the proven engine into a clean, standalone Python library, and then wrapped that library in a Python worker designed to consume jobs from a Redis stream.
6. **The Web Frontend:** Built the Next.js frontend strictly as a trigger-and-render layer, streaming the live trace and ledger updates from Redis and Postgres via Server-Sent Events (SSE).

Trade-offs I made

Decision	Chose	Gained	Negative
Investigation strategy	Hypothesis-first abductive loop, not evidence-first summarization	Compares causes; seeks disconfirming evidence; rejects herrings	Misses the cause if the initial hypothesis set is weak
Agent shape	One bounded investigator, not multi-agent debate	Cheaper, inspectable; one attributable trace	No independent skeptic agent yet
Model	Cheap default (`gemini-3.1-flash-lite`), not a frontier model	Low cost/latency; the diagnosis comes from the harness	Prompt-sensitive.
Human signal	Slack/tickets as low-trust evidence, not a hypothesis seed	War-room consensus can't anchor the model	May underuse a real human clue until it's searched for
Evidence scope	Time-windowed, not global corpus reading	No hindsight leakage; bounded to what responders knew	May exclude older context unless modeled as standing knowledge
Retrieval & graphs	Source-attached tools + lightweight graphs, not RAG or a graph DB	Precise, verifiable citations; minimal infra	No fuzzy cross-incident recall yet

What I would improve with more time

Generalize beyond one synthetic workspace (the #1 risk). Everything is tuned on one company and one cheap model; the fix is a broader eval set (more workspaces, multi-cause incidents, missing telemetry, stale runbooks, conflicting reports, "no incident" cases).

1. **Blend reasoning approaches –** The current hypothesis driven workflow uses abductive reasoning, which does yield promising results, but one of the downfalls is it potentially misses symptoms / knock on causes which are present in the logs but not represented in the initial hypothesis set.

   It would be interesting to try other reasoning approaches. An *inductive* pass (summarize the whole window to surface anomalies bottom-up) and *deductive* checks (apply known runbook rules) would complement it - I suspect a blended approach is best.

2. **Ingest pipeline + evidence warehouse -** Connectors that normalize data sources (e.g. logs, metrics, deploys, alerts, tickets, Slack, topology, and runbooks) into a unified data model (entity resolution, timestamp normalization, dedupe, faster queries). This is the moat – the data.

3. **Evidence handling -** Semantic retrieval would be interesting for cross-incident recall where vocabulary mismatch beats keywords – in addition semantic search would likely work better for unstructured inputs (e.g. chat logs, etc).
4. **Scale / context limits –** The current approach uses time-boxed tools to read logs / files / etc – which will likely be fine given context window sizes available with most models now (1m tokens), but it's not infeasible we are sending huge log files to the LLM, and we quickly fill up the available context. A better approach may be to summarise findings on the fly, spin up sub-agents where appropriate in the investigate loop.
5. **Feedback loops –** The agent could have an additional stage where it can test hypotheses by integrating with code-based frameworks – e.g. the engine could start a Claude code thread to go and interact with the code base to reproduce the identified root cause; then attempt to remediate on-the-fly.